
Semantic Job Matching System → Backend (FastAPI)

Objective

Build a **backend-only** FastAPI service that:

- lets us upload **job descriptions**
- lets us upload **resumes**
- suggests the best-matching jobs for a resume using **embeddings + vector search**
- uses an **LLM** to extract structured info from resumes and to **verify/rerank** the matches

No UI/frontend needed.

Requirements

1) Two main API groups only

You must implement only these two route groups:

- **/jobs** → for job description upload + management
- **/resumes** → for resume upload + matching

(You can add /health if you want.)

Endpoints to implement

Jobs

- POST /jobs
 - Upload JD file (pdf/docx/txt) OR accept raw text
 - Store original text + metadata
 - Chunk + embed + store vectors
- GET /jobs
 - List uploaded jobs (id + basic metadata)
- GET /jobs/{job_id}
 - Return details (metadata + maybe a snippet)
- DELETE /jobs/{job_id}

- Delete job + remove its vectors

Resumes

- POST /resumes
 - Upload resume file (pdf/docx/txt)
 - Extract text
 - Use an LLM to extract structured JSON:
 - skills, projects, work experience, education (keep it minimal)
 - Chunk + embed + store vectors
- GET /resumes/{resume_id}
 - Return extracted structured JSON + metadata
- POST /resumes/{resume_id}/match
 - Vector search against job description vectors
 - Return top job suggestions (ranked)
 - Use an LLM after retrieval to verify/rerank + generate short explanation

Storage (keep it simple)

- Use a **relational DB** (SQLite is fine) for metadata and raw text
- Use a **vector store** (Chroma/Qdrant/FAISS — choose one) for embeddings

Deliverables

1. Running FastAPI project with clear structure (routers/services/models)
2. README with:
 - setup steps
 - required env vars
 - example curl requests
3. Basic tests OR a simple demo script that:
 - uploads a few jobs

- uploads a resume
 - calls match and prints results
-

Hints for tricky parts (don't ignore these)

Parsing files

- PDFs can be messy. Implement parsing so it **fails gracefully**:
 - return a helpful error if extraction is empty/garbled
 - log the issue

Chunking

- Don't blindly embed giant text.
- Chunk JDs and resumes (simple paragraph-based or fixed-size chunks is OK).
- Save chunk metadata (job_id/resume_id + chunk index) so you can trace results.

LLM structured extraction

- Force JSON output and validate it using **Pydantic**.
- If the LLM returns invalid JSON:
 - retry once with a stricter prompt, or
 - run a "repair" pass (simple approach is fine)

Matching quality

- Do **two-stage matching**:
 1. vector search top ~20 jobs
 2. LLM rerank/verify only those top results (don't run LLM on all jobs)

Observability

- Add logging around:
 - file parsing
 - chunk count
 - embedding calls
 - vector search results

- LLM calls
 - This matters more than you think.
-

Definition of Done

I should be able to:

1. upload 10 job descriptions
 2. upload 1 resume
 3. call POST /resumes/{id}/match
...and get back a ranked list of jobs with a short reason for each.
-